

```
In [1]: from textblob import TextBlob
import nltk
nltk.download('punkt')
nltk.download('stopwords') # For removing common words
nltk.download('averaged_perceptron_tagger') # For Part-of-Speech tagging (used i
nltk.download('wordnet') # For Lemmatization
! python -m textblob.download_corpora
# Sample reviews with associated sentiment (as provided)
```

```
[nltk_data] Downloading package punkt to
[nltk_data] C:\Users\Adrien\AppData\Roaming\nltk_data...
[nltk_data] Unzipping tokenizers\punkt.zip.
[nltk_data] Downloading package stopwords to
[nltk_data] C:\Users\Adrien\AppData\Roaming\nltk_data...
[nltk_data] Unzipping corpora\stopwords.zip.
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data] C:\Users\Adrien\AppData\Roaming\nltk_data...
[nltk_data] Unzipping taggers\averaged_perceptron_tagger.zip.
[nltk_data] Downloading package wordnet to
[nltk_data] C:\Users\Adrien\AppData\Roaming\nltk_data...
Finished.
```

```
[nltk_data] Downloading package brown to
[nltk_data] C:\Users\Adrien\AppData\Roaming\nltk_data...
[nltk_data] Unzipping corpora\brown.zip.
[nltk_data] Downloading package punkt_tab to
[nltk_data] C:\Users\Adrien\AppData\Roaming\nltk_data...
[nltk_data] Unzipping tokenizers\punkt_tab.zip.
[nltk_data] Downloading package wordnet to
[nltk_data] C:\Users\Adrien\AppData\Roaming\nltk_data...
[nltk_data] Package wordnet is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data] C:\Users\Adrien\AppData\Roaming\nltk_data...
[nltk_data] Unzipping taggers\averaged_perceptron_tagger_eng.zip.
[nltk_data] Downloading package conll2000 to
[nltk_data] C:\Users\Adrien\AppData\Roaming\nltk_data...
[nltk_data] Unzipping corpora\conll2000.zip.
[nltk_data] Downloading package movie_reviews to
[nltk_data] C:\Users\Adrien\AppData\Roaming\nltk_data...
[nltk_data] Unzipping corpora\movie_reviews.zip.
```

```
In [2]: import pandas as pd

url = "https://raw.githubusercontent.com/pycaret/pycaret/master/datasets/amazon.

print("Téléchargement des 20 000 avis depuis GitHub... ⌚")
df = pd.read_csv(url)

print("Données Amazon chargées avec succès !")
display(df)
```

Téléchargement des 20 000 avis depuis GitHub... ⌚
Données Amazon chargées avec succès !

	reviewText	Positive
0	This is a one of the best apps acording to a b...	1
1	This is a pretty good version of the game for ...	1
2	this is a really cool game. there are a bunch ...	1
3	This is a silly game and can be frustrating, b...	1
4	This is a terrific game on any pad. Hrs of fun...	1
...	...	...
19995	this app is fricken stupid.it froze on the kin...	0
19996	Please add me!!!! I need neighbors! Ginger101...	1
19997	love it! this game. is awesome. wish it had m...	1
19998	I love love love this app on my side of fashio...	1
19999	This game is a rip off. Here is a list of thin...	0

20000 rows × 2 columns

1. Can you perform a sentiment analysis on the collection of product reviews, classify each one as 'positive', 'negative', or 'neutral', and then generate a bar chart to visualize the distribution of these sentiments?"
2. convert the text to lowercase, remove punctuation and numbers, filter out common stopwords, and then lemmatize the remaining words using their appropriate part-of-speech tags.
3. fter the text has been fully preprocessed and lemmatized, could you calculate the frequency of each unique word and then display a bar graph showing the top 15 most common words?
4. generate a word cloud based on the entire corpus of lemmatized review words to provide a visual summary of the most prominent terms.
5. Visualize the frequency of nouns, verbs, adjectives in this text.

```
In [5]: # 1. Analyse des Sentiments
from textblob import TextBlob
import matplotlib.pyplot as plt
import seaborn as sns

def get_sentiment(text):
    # Sécurité au cas où il y a des cellules vides
    if not isinstance(text, str):
        return 'neutral'

    # TextBlob analyse le sentiment (Polarity va de -1 à 1)
    polarity = TextBlob(text).sentiment.polarity
    if polarity > 0:
```

```

        return 'positive'
    elif polarity < 0:
        return 'negative'
    else:
        return 'neutral'

print("Analyse des sentiments en cours...")
df['Sentiment'] = df['reviewText'].apply(get_sentiment)

# Création du graphique à barres
plt.figure(figsize=(8, 5))
sns.countplot(data=df, x='Sentiment', hue='Sentiment', palette='viridis', order=
plt.title('Distribution des Sentiments des Clients')
plt.xlabel('Sentiment')
plt.ylabel('Nombre d\'avis')
plt.show()

# 2. Pipeline de Prétraitement NLP
import re
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from nltk.corpus import wordnet

lemmatizer = WordNetLemmatizer()
stop_words = set(stopwords.words('english'))

# Convertir les catégories grammaticales pour Le Lemmatizer
def get_wordnet_pos(treebank_tag):
    if treebank_tag.startswith('J'):
        return wordnet.ADJ
    elif treebank_tag.startswith('V'):
        return wordnet.VERB
    elif treebank_tag.startswith('N'):
        return wordnet.NOUN
    elif treebank_tag.startswith('R'):
        return wordnet.ADV
    else:
        return wordnet.NOUN

def preprocess_text(text):
    if not isinstance(text, str):
        return ""
    # a. Minuscules
    text = text.lower()
    # b. Suppression de la ponctuation et des chiffres
    text = re.sub(r'^a-z\s', '', text)
    # Tokenisation
    words = text.split()
    # c. POS Tagging
    pos_tags = nltk.pos_tag(words)

    clean_words = []
    # d. Stopwords et Lemmatisation
    for word, tag in pos_tags:
        if word not in stop_words:
            wn_pos = get_wordnet_pos(tag)
            lemma = lemmatizer.lemmatize(word, pos=wn_pos)
            clean_words.append(lemma)

```

```

        return " ".join(clean_words)

print("Nettoyage intensif en cours")
df['Clean_Text'] = df['reviewText'].apply(preprocess_text)
print("Nettoyage terminé !")
df[['reviewText', 'Clean_Text']].head()

# 3. Fréquence des mots
from collections import Counter

# On rassemble tous les mots nettoyés dans une grande liste
all_words = " ".join(df['Clean_Text']).split()
word_counts = Counter(all_words)

# On garde les 15 premiers
common_words = word_counts.most_common(15)
words_df = pd.DataFrame(common_words, columns=['Mot', 'Frequence'])

# Création du graphique
plt.figure(figsize=(10, 6))
sns.barplot(data=words_df, x='Frequence', y='Mot', hue='Mot', palette='mako', legend=False)
plt.title('Top 15 des mots les plus fréquents (après nettoyage)')
plt.xlabel('Fréquence d\'apparition')
plt.ylabel('Mots')
plt.show()

# 4. Génération du Nuage de Mots
from wordcloud import WordCloud

# On regroupe tout le texte nettoyé
all_text_combined = " ".join(df['Clean_Text'])

# Création du nuage
wordcloud = WordCloud(width=800, height=400, background_color='white', colormap=

plt.figure(figsize=(12, 6))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title('Nuage de Mots des Avis Clients')
plt.show()

# 5. Fréquence des catégories grammaticales (POS)
print("Comptage des Noms, Verbes et Adjectifs en cours...")

def count_pos(text):
    if not isinstance(text, str):
        return 0, 0, 0
    # Tokenisation rapide
    words = text.split()
    tags = nltk.pos_tag(words)

    # Comptage via les tags NLTK (N=Nom, V=Verbe, J=Adjectif)
    n = sum(1 for word, tag in tags if tag.startswith('N'))
    v = sum(1 for word, tag in tags if tag.startswith('V'))
    a = sum(1 for word, tag in tags if tag.startswith('J'))
    return n, v, a

# On applique la fonction et on additionne les totaux
pos_counts = df['Clean_Text'].apply(count_pos)

```

```

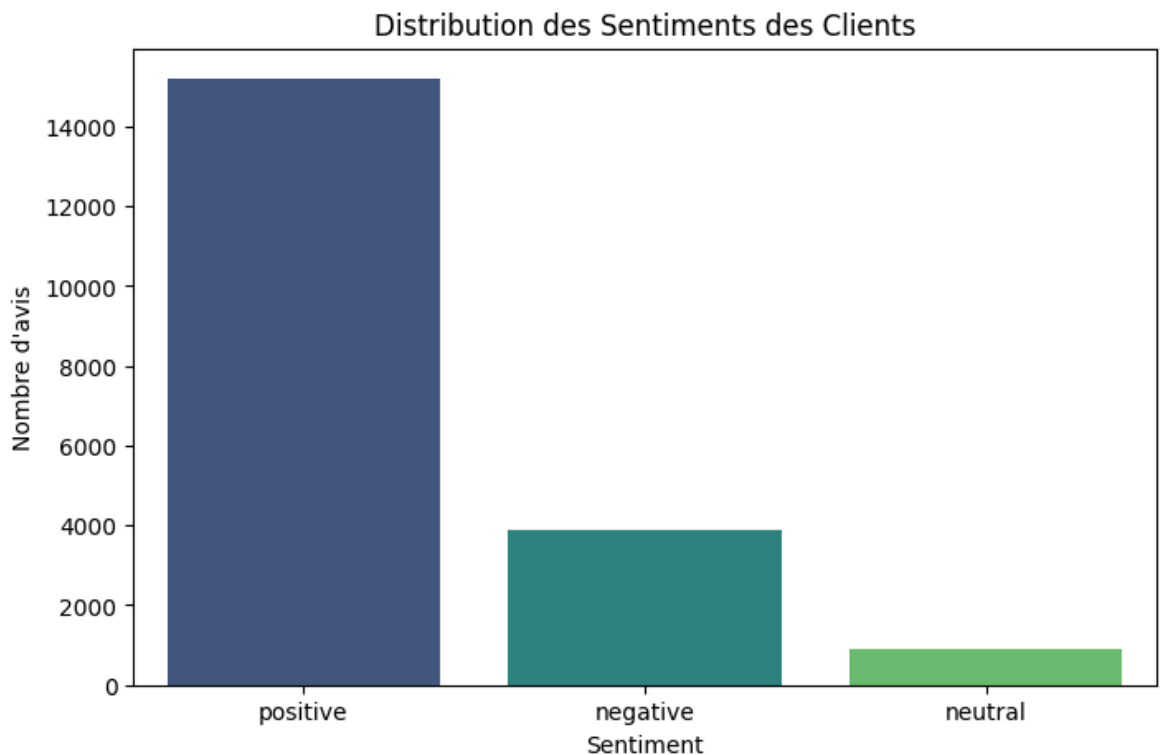
noun_total = sum(x[0] for x in pos_counts)
verb_total = sum(x[1] for x in pos_counts)
adj_total = sum(x[2] for x in pos_counts)

# Création du DataFrame pour le graphique
pos_df = pd.DataFrame({
    'Type': ['Noms (Nouns)', 'Verbes (Verbs)', 'Adjectifs (Adjectives)'],
    'Frequence': [noun_total, verb_total, adj_total]
})

plt.figure(figsize=(8, 5))
sns.barplot(data=pos_df, x='Type', y='Frequence', hue='Type', palette='autumn',
plt.title('Distribution des Noms, Verbes et Adjectifs')
plt.ylabel('Nombre total d\'apparitions')
plt.show()

```

Analyse des sentiments en cours...



Nettoyage intensif en cours
Nettoyage terminé !

A horizontal bar chart titled 'Mots' on the y-axis and 'Fréquence d'apparition' on the x-axis. The y-axis lists 15 words: app, game, get, use, love, like, great, play, work, time, kindle, one, fun, good, and dont. The x-axis has major ticks at 0, 2000, 4000, 6000, 8000, and 10000. The bars are colored in a gradient from dark purple for 'app' to light green for 'dont'. The approximate frequencies are: app (10800), game (6800), get (5100), use (4900), love (4600), like (4200), great (4100), play (3400), work (3300), time (2900), kindle (2900), one (2800), fun (2600), good (2500), and dont (2400).

Mots	Fréquence d'apparition
app	10800
game	6800
get	5100
use	4900
love	4600
like	4200
great	4100
play	3400
work	3300
time	2900
kindle	2900
one	2800
fun	2600
good	2500
dont	2400

[illegible]

Comptage des Noms, Verbes et Adjectifs en cours...

